CS779: Advanced Database Management

Final Term Project Report

Sharmila Gopal

CourseForge: An Online Learning Platform Data Warehouse

1. Finalized Proposal

1.1. What is the advanced database area you are focusing on related to this course?

Data warehousing and OLAP (Online Analytical Processing), specifically dimensional modeling (star schema design) and advanced analytical SQL functions such as ROLLUP, CUBE, and window functions. This builds directly on OLAP material covered in CS779 - CUBE was already used in this course to compute appointment subtotals and grand totals across marital status and ethnicity for the Terrier Hospital database. This project uses an original domain - CourseForge, an online learning platform - rather than the InternMatch scenario from Assignment 1 and Assignment 4, so that both the OLTP schema and the star schema represent original design work for this course rather than a reuse of prior assignments.

Following facilitator feedback, the OLTP schema was expanded to 20 tables to better reflect the scope and complexity expected at the Assignment 1 level, and both ERDs use proper crow's-foot notation.

1.2. What are you planning to implement? (proof of concept component)

- A normalized OLTP schema (20 tables) built specifically for this project
- A star schema data warehouse, with fact_quiz_attempt as the central fact table - grained at one row per quiz attempt, carrying five measures (score, max_score, time_spent_seconds, attempt_number, pass_flag) - surrounded by dim_student, dim_course, dim_instructor, dim_quiz, and dim_date
- An automated OLTP -> DW pipeline built using SQL stored procedures; Python is used only for the initial bulk load of synthetic CourseForge data into the OLTP schema

- Type 2 Slowly Changing Dimension (SCD) handling on dim_course, since course price and level are edited over time by instructors
- A set of analytical SQL queries using ROLLUP, CUBE, and window functions
- A Power BI dashboard connected directly to the data warehouse to visualize the results

1.3. What are some of the goals that you plan to learn in this project?

- Understand the practical difference between OLTP and OLAP schema design, and when each is appropriate
- Learn dimensional modeling: fact vs. dimension tables, grain, and designing a schema for analytical performance rather than transactional integrity
- Build a real, SQL-automated pipeline that moves and reshapes data between two different schema paradigms
- Learn to design and implement Slowly Changing Dimensions (Type 2) to preserve historical accuracy in analytical reporting
- Apply advanced SQL analytics (window functions, CUBE/ROLLUP) where they're the correct tool for the job, across a fact table with multiple measures

Gain hands-on experience connecting a database to a professional BI tool (Power BI)

1.4. What skills are you bringing from other courses, and what is the new element you're learning in this class related to advanced database management?

From CS669 (Database Design and Implementation for Business), a solid foundation in relational schema design, ER modeling, and normalization is applied to building the CourseForge OLTP schema from scratch. From Data Science with Python, the ability to build data-loading scripts is applied to the initial OLTP load. From Foundations of Analytics and Visualization, the ability to turn query results into meaningful charts/dashboards supports the Power BI component.

The genuinely new elements for this course are: (1) dimensional modeling and OLAP-specific database design; (2) automating the OLTP-to-DW transformation using SQL stored procedures rather than external ETL tooling; (3) designing and implementing Slowly Changing Dimensions on dim_course; and (4) building a fact table with multiple measures that supports several distinct analytical questions rather than a single metric.

1.5. What data are you looking to use specifically?

A CourseForge scenario: an online learning platform where students enroll in instructor-authored courses, complete modules and lessons, take quizzes, and make payments. A synthetic dataset is generated from scratch - enough students, courses, and quiz attempts to produce meaningful ROLLUP, CUBE, and window function results, including a course price change to exercise the Type 2 SCD logic on dim_course. A deliberately messy/dirty raw data sample is also included (Section 2.4) to demonstrate data-quality issues that normalization and cleaning steps are designed to catch.

2. Draft Design Components

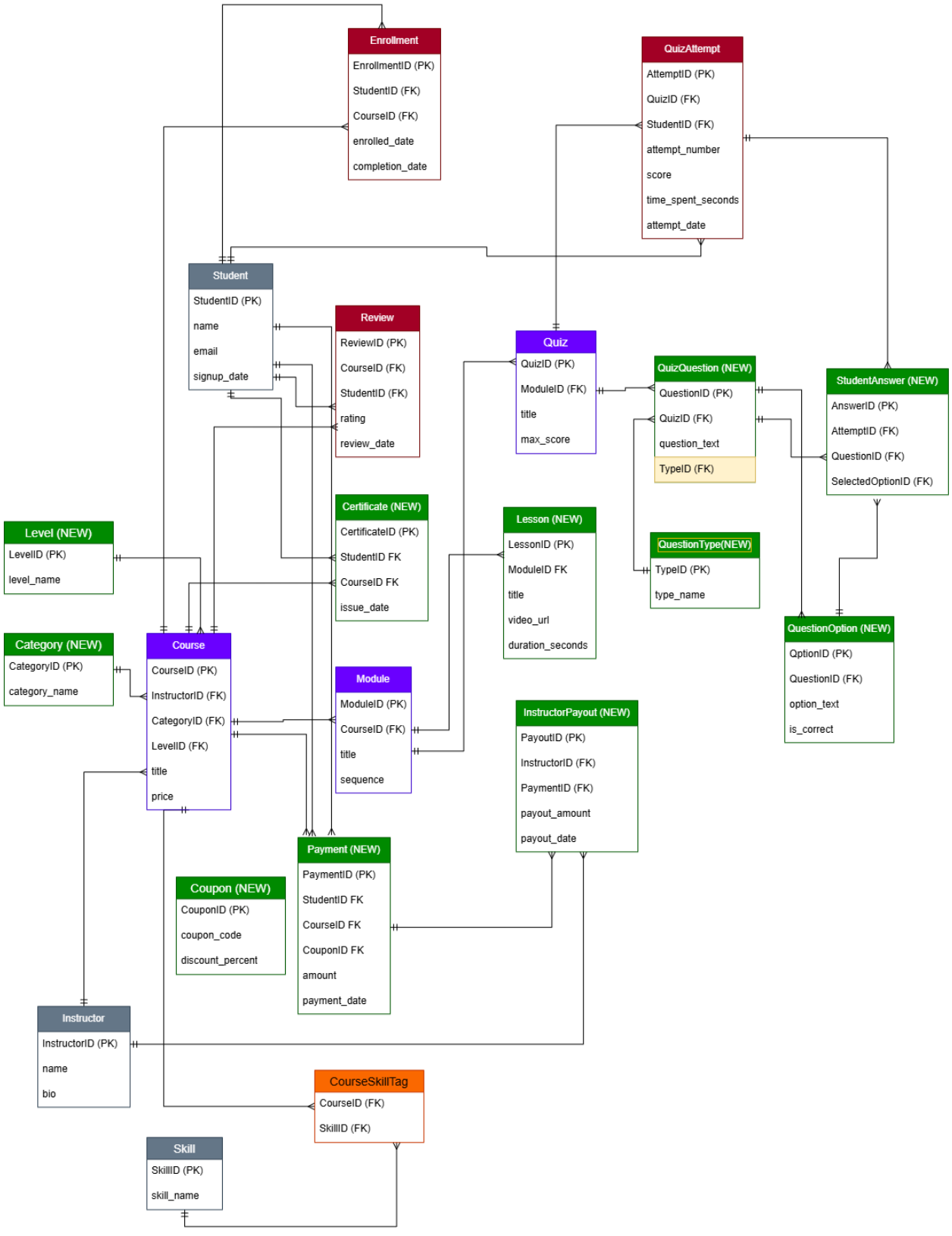
2.1 OLTP Source Schema

The OLTP schema is a normalized design for CourseForge, using crow's-foot entity relationships, spanning 20 tables across course content (Course, Module, Lesson), assessment (Quiz, QuizQuestion, QuestionOption, QuizAttempt, StudentAnswer), commerce (Payment, Coupon, InstructorPayout, Certificate), and supporting entities (Student, Instructor, Category, Level, Skill, CourseSkillTag, Enrollment, Review).

Table	Key Columns
Student	**StudentID (PK), name, email, signup_date**
Instructor	**InstructorID (PK), name, bio**
Category (NEW)	CategoryID (PK), category_name
Level (NEW)	LevelID (PK), level_name
Course	**CourseID (PK), InstructorID (FK), title, category, price, level**
Module	**ModuleID (PK), CourseID (FK), title, sequence**
Lesson (NEW)	LessonID (PK), ModuleID (FK), title, video_url, duration_seconds
Quiz	**QuizID (PK), ModuleID (FK), title, max_score**
QuestionType (NEW)	TypeID (PK), type_name
QuizQuestion (NEW)	QuestionID (PK), QuizID (FK), question_text, TypeID (FK)

QuestionOption (NEW)	OptionID (PK), QuestionID (FK), option_text, is_correct
Enrollment	**EnrollmentID (PK), StudentID (FK), CourseID (FK), enrolled_date, completion_date**
QuizAttempt	**AttemptID (PK), QuizID (FK), StudentID (FK), attempt_number, score, time_spent_seconds, attempt_date**
StudentAnswer (NEW)	AnswerID (PK), AttemptID (FK), QuestionID (FK), SelectedOptionID (FK)
Review	**ReviewID (PK), CourseID (FK), StudentID (FK), rating, review_date**
Skill	**SkillID (PK), skill_name**
CourseSkillTag	**CourseID (FK), SkillID (FK) - junction table**
Coupon (NEW)	CouponID (PK), coupon_code, discount_percent
Payment (NEW)	PaymentID (PK), StudentID (FK), CourseID (FK), CouponID (FK, nullable), amount, payment_date
Certificate (NEW)	CertificateID (PK), StudentID (FK), CourseID (FK), issue_date
InstructorPayout (NEW)	PayoutID (PK), InstructorID (FK), PaymentID (FK), payout_amount, payout_date

Full OLTP entity-relationship diagram (crow's-foot notation):

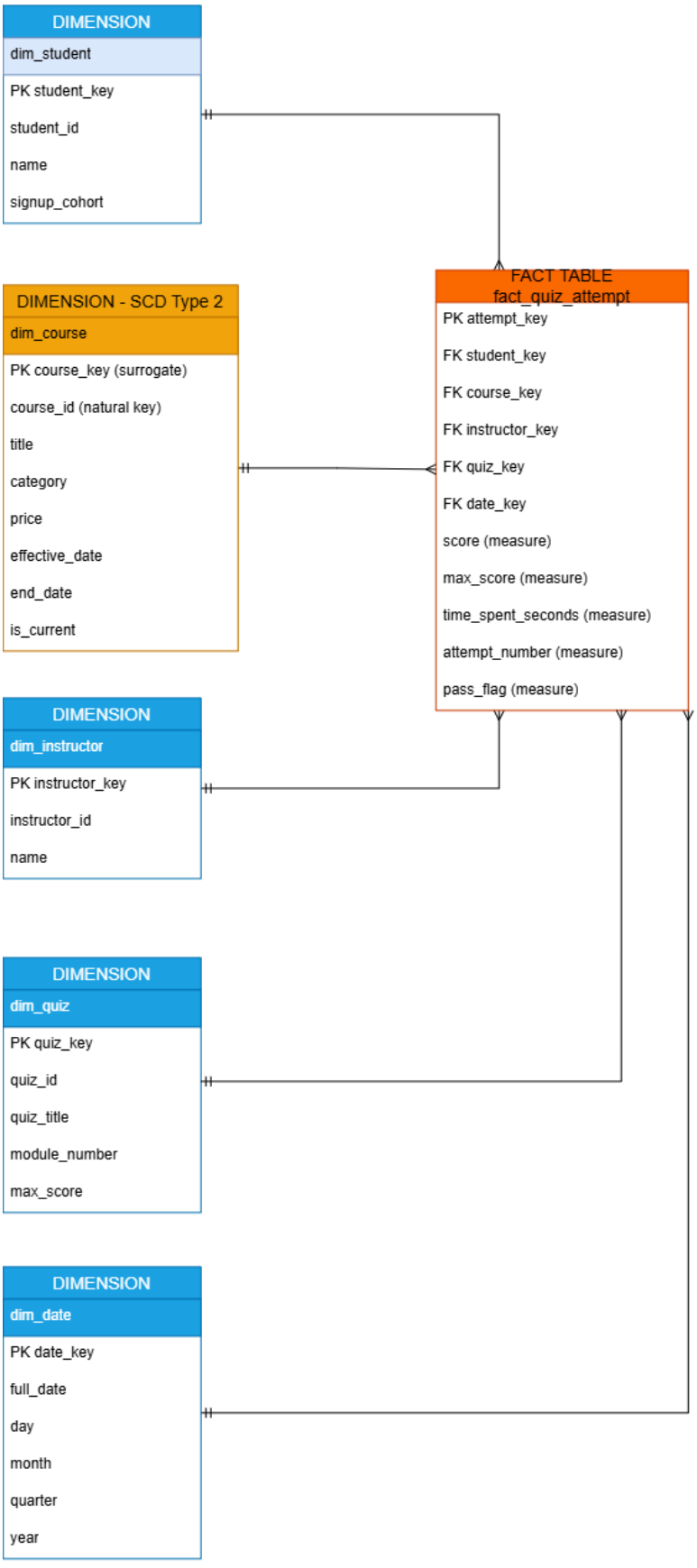

Enrollment
EnrollmentID (PK)
StudentID (FK)
CourseID (FK)
enrolled_date
completion_date
QuizAttempt
AttemptID (PK)
QuizID (FK)
StudentID (FK)
attempt_number
score
time_spent_seconds
attempt_date
Student
StudentID (PK)
name
email
signup_date
Review
ReviewID (PK)
CourseID (FK)
StudentID (FK)
rating
review_date
Quiz
QuizID (PK)
ModuleID (FK)
title
max_score
QuizQuestion (NEW)
QuestionID (PK)
QuizID (FK)
question_text
TypeID (FK)
StudentAnswer (NEW)
AnswerID (PK)
AttemptID (FK)
QuestionID (FK)
SelectedOptionID (FK)
Certificate (NEW)
CertificateID (PK)
StudentID FK
CourseID FK
issue_date
Lesson (NEW)
LessonID (PK)
ModuleID FK
title
video_url
duration_seconds
QuestionType(NEW)
TypeID (PK)
type_name
Level (NEW)
LevelID (PK)
level_name
QuestionOption (NEW)
QptionID (PK)
QuestionID (FK)
option_text
is_correct
Category (NEW)
CategoryID (PK)
category_name
Course
CourseID (PK)
InstructorID (FK)
CategoryID (FK)
LevelID (FK)
title
price
Module
ModuleID (PK)
CourseID (FK)
title
sequence
InstructorPayout (NEW)
PayoutID (PK)
InstructorID (FK)
PaymentID (FK)
payout_amount
payout_date
Payment (NEW)
PaymentID (PK)
StudentID FK
CourseID FK
CouponID FK
amount
payment_date
Coupon (NEW)
CouponID (PK)
coupon_code
discount_percent
Instructor
InstructorID (PK)
name
bio
CourseSkillTag
CourseID (FK)
SkillID (FK)
Skill
SkillID (PK)
skill_name

2.2 Data Warehouse Star Schema

The star schema dimensionalizes the CourseForge OLTP data for analytical querying. fact_quiz_attempt is grained at one row per quiz attempt and carries five measures. dim_course is modeled as a Type 2 Slowly Changing Dimension to preserve historical accuracy when course price or category changes over time.

Table	Type	Key Columns
fact_quiz_attempt	Fact	attempt_key (PK), student_key (FK), course_key (FK), instructor_key (FK), quiz_key (FK), date_key (FK), score, max_score, time_spent_seconds, attempt_number, pass_flag
dim_student	Dimension	student_key (PK), student_id (natural key), name, signup_cohort
dim_course	Dimension (SCD Type 2)	course_key (surrogate PK), course_id (natural key), title, category, price, effective_date, end_date, is_current
dim_instructor	Dimension	instructor_key (PK), instructor_id (natural key), name
dim_quiz	Dimension	quiz_key (PK), quiz_id (natural key), quiz_title, module_number, max_score
dim_date	Dimension	date_key (PK), full_date, day, month, quarter, year

Full star schema entity-relationship diagram, with FACT TABLE and SCD Type 2 explicitly labeled:


DIMENSION
dim_student
PK student_key
student_id
name
signup_cohort
DIMENSION - SCD Type 2
dim_course
PK course_key (surrogate)
course_id (natural key)
title
category
price
effective_date
end_date
is_current
FACT TABLE
fact_quiz_attempt
PK attempt_key
FK student_key
FK course_key
FK instructor_key
FK quiz_key
FK date_key
score (measure)
max_score (measure)
time_spent_seconds (measure)
attempt_number (measure)
pass_flag (measure)
DIMENSION
dim_instructor
PK instructor_key
instructor_id
name
DIMENSION
dim_quiz
PK quiz_key
quiz_id
quiz_title
module_number
max_score
DIMENSION
dim_date
PK date_key
full_date
day
month
quarter
year

2.3 Normalization Example: 3NF and BCNF

Before normalization (3NF violation): category and level are stored as repeating text directly on Course, creating redundancy and a transitive dependency.

CourseID	title	category	level	InstructorID
1	Intro to SQL	Data Science	Beginner	3
2	Advanced SQL	Data Science	Advanced	3
3	Guitar Basics	Music	Beginner	7

After normalization (3NF): category and level are extracted into their own lookup tables, referenced by foreign key.

CourseID	title	CategoryID	LevelID	InstructorID
1	Intro to SQL	1	1	3
2	Advanced SQL	1	2	3
3	Guitar Basics	2	1	7

Category lookup:

CategoryID	category_name
1	Data Science
2	Music

Level lookup:

LevelID	level_name
1	Beginner
2	Advanced

BCNF example: Coupon. In a denormalized Payment table, coupon_code would determine discount_percent, but coupon_code is not a candidate key of Payment (the same code can be reused across many payments) - this violates BCNF unless Coupon is split into its own table with CouponID as the true key.

CouponID	coupon_code	discount_percent
101	WELCOME10	10
102	SUMMER25	25
103	WELCOME10	10

A related correction, identified through Q&A after the initial presentation: QuizQuestion originally stored question_type as repeating text directly on the table, the same pattern corrected above for category and level. This has since been fixed by extracting a QuestionType lookup table (TypeID, type_name), with QuizQuestion referencing it via TypeID (see Section 2.1).

2.4 Messy / Dirty Sample Data

A deliberately dirty raw-data sample, used to illustrate the data-quality issues that a staging/cleaning step before loading into the normalized OLTP schema must catch:

raw_id	name	email	signup_date
1	Sarah Mitchell	Sarah.Mitchell@gmail.com	2026-01-15
2	sarah mitchell	sarah.mitchell@GMAIL.com	01/15/2026
3	James O Connor	jamesoconnor@yahoo.com	(blank)
4	James O'Connor	JAMESOCONNOR@yahoo.com	2026-02-3
5	(blank)	priya.patel@outlook.com	2026/03/01

- Rows 1-2: same real person, inconsistent name capitalization and email casing, and two different date formats for the same date.
- Row 3: extra internal whitespace in the name, and a missing signup_date.
- Row 4: apostrophe in the name ("O'Connor") that must be handled carefully in text processing and SQL string escaping.
- Row 5: missing name entirely, and a third distinct date format (YYYY/MM/DD).

2.5 Sample Data (Clean, Modeled)

Original sample data illustrating the finalized design (synthetic, generated for this project):

OLTP - QuizAttempt

AttemptID	QuizID	StudentID	attempt_number	score	time_spent_seconds	attempt_date
501	12	8	1	62	340	2026-06-01
502	12	8	2	88	210	2026-06-02
503	12	15	1	95	180	2026-06-03

DW - dim_course (SCD Type 2, showing a price change)

course_key	course_id	title	price	effective_date	end_date	is_current
41	C-104	SQL for Analysts	49.00	2026-01-01	2026-05-31	0
92	C-104	SQL for Analysts	59.00	2026-06-01	NULL	1

3. Project Plan

Dates	Task	Deliverable / Milestone
Jul 28 – Aug 1	Finalize CourseForge OLTP + star schema; generate synthetic dataset	Finalized schemas + populated synthetic dataset
Aug 2	Submit Term Project Update #3; incorporate facilitator feedback: expand to 20 tables, crow's-foot notation, 3NF/BCNF example, messy data sample, labeled star schema	Update #3 submitted
Aug 3 – Aug 6	Load expanded OLTP schema into PostgreSQL; write and test SQL stored procedures for the ETL pipeline, including Type 2 SCD logic for dim_course	Working ETL stored procedures + SCD-enabled dimension load
Aug 7 – Aug 10	Write and validate analytical SQL queries (ROLLUP, CUBE, window functions);	Query set + working Power BI dashboard

	connect Power BI and build dashboard visuals	
Aug 11 – Aug 12	End-to-end testing and refinement of the full pipeline; prepare and rehearse live demo	Stable, tested pipeline
Aug 12	Live project demo presentation	Live demo to be delivered
Aug 13 – Aug 15	Incorporate any demo feedback; write delta report covering changes and progress since live presentation	Draft delta report
Aug 16	Final review, polish, and submission	Delta report submission

4. Implementation and Results

4.1 Data Generation

Synthetic data for the full 20-table CourseForge schema was generated using a Python script built on the Faker library. The generator produces realistic values for every entity - student names and emails, course titles and prices, quiz questions and answers, payments and coupon usage - while enforcing correct foreign key relationships throughout, so every generated row references a real, valid parent record.

The final dataset totals approximately 7,340 rows across 20 OLTP tables and 6 data warehouse tables, including 150 students, 30 courses, 90 quizzes, 450 quiz questions, 600 quiz attempts, 1,800 recorded student answers, and 229 completed payments. The generator also deliberately engineers one Type 2 SCD event - a price change on a specific course - so the pipeline's history-tracking logic has a real, testable case to handle, rather than a purely theoretical one.

4.2 Automated Pipeline: Stored Procedures

The OLTP-to-warehouse transformation is fully automated through seven PostgreSQL stored procedures, avoiding any manual or external ETL tooling:

- Four procedures each load a simple dimension table: dim_student, dim_instructor, dim_quiz, and dim_date.

- sp_load_dim_course_scd2 is the most important: for every course, it compares the current OLTP values against the warehouse's active row for that course. If the price or category has changed, it expires the old row (setting end_date and is_current = FALSE) and inserts a brand-new row with a new surrogate key.
- sp_load_fact_quiz_attempt loads the fact table, resolving each quiz attempt to the correct historical version of dim_course based on the attempt's own date - not simply whichever version is current today.
- sp_run_full_etl runs all six procedures in the correct dependency order with a single CALL statement.

Live proof the SCD2 logic works, not just runs: Course #1's price was changed directly in the OLTP Course table (from $41.99 to $79.99), and sp_load_dim_course_scd2 was re-run. The result, queried directly from dim_course:

	course_key [PK] integer	course_id integer	price numeric (6,2)	effective_date date	end_date date	is_current boolean
1	1	1	41.99	2000-01-01	2026-08-05	false
2	31	1	79.99	2026-08-06	[null]	true

The original row (course_key 1) is correctly expired - its end_date is now set and is_current is false. A brand-new row (course_key 31) was automatically created with the new price, effective today, and marked as the current version. This confirms the pipeline preserves history rather than silently overwriting it, which is the entire purpose of Type 2 SCD.

A real debugging note: the first version of this pipeline produced zero rows in fact_quiz_attempt on its initial run. Investigation showed that newly-inserted dim_course rows were being stamped with effective_date = CURRENT_DATE, while all historical quiz attempts occurred before today - so no fact row could find a matching dimension version. The fix was to backdate a brand-new course's initial effective_date to the start of recorded history rather than to today, reserving CURRENT_DATE specifically for genuine, detected changes. This reflects a real, non-obvious pitfall in SCD2 design: the first load of a dimension needs different date-handling logic than a subsequent change event.

4.3 Analytical Queries: ROLLUP, CUBE, and Window Functions

Five analytical queries were written and run against the populated warehouse, covering ROLLUP, CUBE, and three different window-function techniques.

Query 1 - ROLLUP: pass rate by category, with an automatic grand-total row.

	category character varying	total_attempts bigint	passed_attempts bigint	pass_rate_pct numeric
1	ALL CATEGORIES	600	415	69.2
2	Business	60	42	70.0
3	Data Science	73	44	60.3
4	Design	66	42	63.6
5	Marketing	113	86	76.1
6	Music	64	42	65.6
7	Photography	43	31	72.1
8	Programming	59	43	72.9
9	Web Development	122	85	69.7

Finding: the overall pass rate is 69.2%. Marketing has the highest pass rate at 76.1%, while Data Science is the most difficult category at 60.3%.

Query 2 - CUBE: average score by category and quarter, giving category-only, quarter-only, and combined subtotals in a single query.

	category character varying	quarter text	attempts bigint	avg_score numeric
1	ALL CATEGORIES	1	91	69.1
2	ALL CATEGORIES	2	344	71.2
3	ALL CATEGORIES	3	165	70.0
4	ALL CATEGORIES	ALL QUARTE...	600	70.6
5	Business	1	9	54.6
6	Business	2	31	71.9
7	Business	3	20	74.8

Total rows: 36 Query complete 00:00:00.074

Finding: average scores are fairly stable across quarters (69–74), with more variation appearing at the category level than over time.

Query 3 - Window function (RANK): ranks instructors by average student score.

	instructor_name character varying (100)	total_attempts bigint	avg_score numeric	score_rank bigint
1	Shannon Hernandez	27	78.4	1
2	Nathan Maldonado	62	72.3	2
3	Jennifer Rocha	104	71.3	3
4	Melissa Robinson	73	71.3	4
5	Sarah Romero	88	70.4	5
6	Allison Hill	61	69.6	6
7	Shannon Ray	40	69.0	7
8	Gregory Baker	22	68.7	8

Total rows: 10 Query complete 00:00:00.130

Finding: Shannon Hernandez has the highest-ranked average score at 78.4, noticeably ahead of the rest of the instructor pool.

Query 4 - Window function (AVG...OVER): compares each time-spent bucket's average score against the overall average without collapsing the detail rows.

	time_bucket	attempts bigint	avg_score_in_bucket numeric	avg_score_overall numeric
1	1. Under 5 m...	155	71.2	70.6
2	2. 5-10 min	204	70.3	70.6
3	3. Over 10 min	241	70.3	70.6

Add row
Alt Shift A

Finding: students who spent under 5 minutes actually scored slightly higher (71.2) than those who spent longer - a mildly counterintuitive result suggesting quiz difficulty, not time invested, is the stronger driver of score in this dataset.

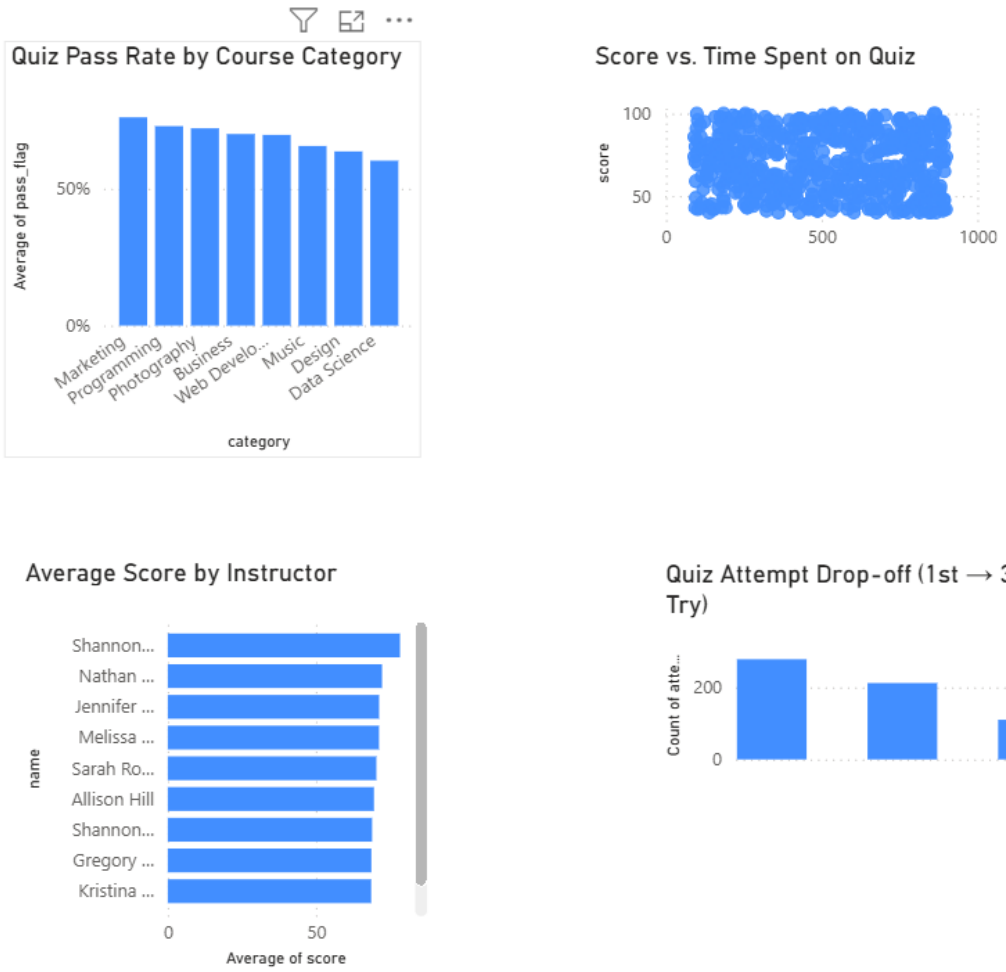
Query 5 - Window function (FIRST_VALUE): tracks attempt-number drop-off as a percentage of first attempts.

	attempt_number integer	attempts_at_this_number bigint	pct_of_first_attempts numeric
1	1	278	100.0
2	2	212	76.3
3	3	110	39.6

Finding: only 76.3% of students who attempt a quiz once try again, and only 39.6% reach a third attempt - a clear drop-off funnel.

4.4 Power BI Dashboard

A Power BI dashboard was connected directly to the six warehouse tables (dim_student, dim_course, dim_instructor, dim_quiz, dim_date, fact_quiz_attempt), with relationships modeled to match the star schema exactly. Four visuals were built, each corresponding to one of the analytical queries above, confirming the dashboard's numbers match the underlying SQL results.



Top-left: quiz pass rate by category, matching Query 1. Top-right: score vs. time spent (scatter of all ~600 attempts), confirming the flat relationship found in Query 4. Bottom-left: average score by instructor, matching Query 3. Bottom-right: the attempt-number drop-off funnel, matching Query 5.

5. Conclusion

This project took a single design decision - building an original schema rather than reusing prior coursework - through a complete, verified pipeline: a 20-table normalized OLTP schema, a five-measure star schema warehouse with a working Type 2 Slowly Changing Dimension, ~7,340 rows of synthetic but referentially-correct data, a fully automated SQL stored-procedure ETL layer, five analytical queries spanning ROLLUP, CUBE, and window functions, and a Power BI dashboard whose results independently confirm the SQL findings.

The most valuable lesson was the practical difference between designing a data warehouse and proving it actually works: the SCD2 logic looked correct on paper well before the date-handling bug surfaced during the first real pipeline run, and only running real data through the real procedures exposed it. That distinction - between a schema that is theoretically sound and a pipeline that is empirically verified - was the central, hands-on takeaway of this project.

6. Sources

Kimball Group. "Dimensional Modeling Techniques." kimballgroup.com.

This is the primary reference for the dimensional modeling methodology this project is built on, covering star schema design, fact and dimension table structure, and grain. It provides the framework for separating measurable facts from descriptive dimensions.

Microsoft Learn. "Understand star schema and the importance for Power BI." learn.microsoft.com.

This resource explains why star schema design matters specifically for Power BI performance and usability, directly relevant since the project's dashboard layer connects to the DW via Power BI.

MSSQLTips. "Using the SQL Server MERGE Statement to Process Type 2 Slowly Changing Dimensions." mssqltips.com.

This source provides a concrete, implementable approach for handling Type 2 SCD logic, which is exactly the mechanism this project's sp_load_dim_course_scd2 procedure implements for dim_course.